LAB9_Shiguihra

Gabriel Furlan Shiguihara

Table of contents

0. Configuração do ambiente **1**
0.1 Biblioteca R: `reticulate` e pacotes Python 1
0.2 Bibliotecas Python 2

1. Importar o conjunto de dados `diamonds` **3**

2. Associação entre peso (`carat`) e preço (`price`) **3**
Parágrafo de achados (Questão 2) 5

3. A relação carat × price é a mesma em todos os cortes? **6**
3.1 Um painel por corte 6
3.2 Todas as retas juntas 7
Parágrafo de conclusão (Questão 3) 8

4. Preço × peso considerando corte, cor e clareza ao mesmo tempo **8**
4.1 Tentativa direta: facetas cor × clareza, retas coloridas por corte 9
4.2 Resumindo a informação: mapa de calor das inclinações 10
4.3 Quanto o patamar de preço muda? (preço mediano por 1 ct) 12
Dificuldades encontradas e conclusão (Questão 4) 13

Informações da sessão (reprodutibilidade) **14**

0. Configuração do ambiente

0.1 Biblioteca R: `reticulate` e pacotes Python

O `reticulate` conecta o R ao Python. A função `py_require()` declara quais pacotes Python são necessários; na primeira execução eles são baixados automaticamente (pode levar alguns minutos).

```r
library(reticulate)

# Declara os pacotes Python usados neste documento (instalados automaticamente)
py_require(c("pandas", "numpy", "plotnine", "statsmodels", "scipy", "matplotlib"))

py_config()
```

```
python:         C:/Users/user/AppData/Local/R/cache/R/reticulate/uv/cache/archive-
v0/89JUTz3byNJbRWVz/Scripts/python.exe
libpython:      C:/Users/user/AppData/Local/R/cache/R/reticulate/uv/python/cpython-
3.13.16-windows-x86_64-none/python313.dll
pythonhome:     C:/Users/user/AppData/Local/R/cache/R/reticulate/uv/cache/archive-
v0/89JUTz3byNJbRWVz
virtualenv:     C:/Users/user/AppData/Local/R/cache/R/reticulate/uv/cache/archive-
v0/89JUTz3byNJbRWVz/Scripts/activate_this.py
version:        3.13.16 (main, Oct  9 2026, 15:21:05) [MSC v.1944 64 bit (AMD64)]
Architecture:   64bit
numpy:          C:/Users/user/AppData/Local/R/cache/R/reticulate/uv/cache/archive-
v0/89JUTz3byNJbRWVz/Lib/site-packages/numpy
numpy_version:  2.5.4

NOTE: Python version was forced by VIRTUAL_ENV
```

0.2 Bibliotecas Python

```python
# pandas: manipulação de tabelas
import pandas as pd

# numpy: operações numéricas (logaritmos, ajuste de retas)
import numpy as np

# plotnine: gráficos em camadas (gramática de gráficos, como o ggplot2)
from plotnine import *
from plotnine.data import diamonds as diamonds_builtin

import warnings
warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 20)
pd.set_option("display.width", 120)
```

1. Importar o conjunto de dados `diamonds`

Se existir um arquivo `diamonds.csv` na mesma pasta deste documento, ele é usado. Caso contrário, usa-se o conjunto `diamonds` embutido no `plotnine`.

```
import os

caminho_csv = "diamonds.csv"

if os.path.exists(caminho_csv):
    diamonds = pd.read_csv(caminho_csv)
    # remove coluna de índice sem nome, se existir
    diamonds = diamonds.loc[:, ~diamonds.columns.str.contains("^Unnamed")]
    print("Dados lidos do arquivo CSV.")
else:
    diamonds = diamonds_builtin.copy()
    print("Arquivo CSV não encontrado: usando plotnine.data.diamonds.")
```

```
Arquivo CSV não encontrado: usando plotnine.data.diamonds.
```

```
print("Linhas, colunas:", diamonds.shape)
```

```
Linhas, colunas: (53940, 10)
```

```
diamonds.head()
```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

2. Associação entre peso (`carat`) e preço (`price`)

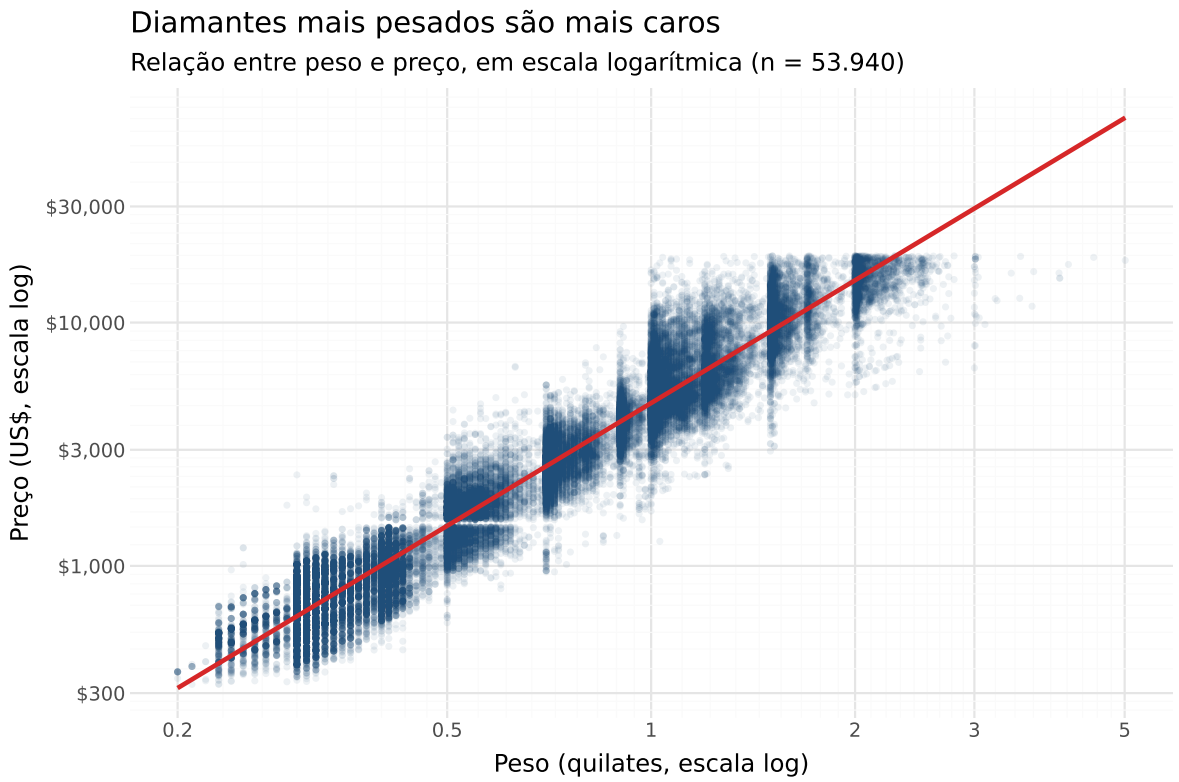
Ações para um gráfico de boa qualidade: transparência (`alpha`) contra a sobreposição de pontos, escala log nos dois eixos para linearizar a relação, linha de tendência e rótulos claros.

```python
# Função personalizada para formatar valores em dólares sem depender do mizani
def dollar_format(x):
    return [f"${v:,.0f}" for v in x]

fmt_dolar = dollar_format

ordem_cut = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
ordem_color = ["D", "E", "F", "G", "H", "I", "J"]
ordem_clarity = ["I1", "SI2", "SI1", "VS2", "VS1", "VVS2", "VVS1", "IF"]

#| label: q2-melhorado
#| fig-width: 7.5
#| fig-height: 5
p_q2 = (
    ggplot(diamonds, aes(x="carat", y="price"))
    + geom_point(alpha=0.08, size=0.8, color="#1f4e79")
    + geom_smooth(method="lm", color="#d62728", size=1.2)
    + scale_x_log10(breaks=[0.2, 0.5, 1, 2, 3, 5])
    + scale_y_log10(labels=dollar_format)
    + labs(
        title="Diamantes mais pesados são mais caros",
        subtitle="Relação entre peso e preço, em escala logarítmica (n = 53.940)",
        x="Peso (quilates, escala log)",
        y="Preço (US$, escala log)",
    )
    + theme_minimal()
    + theme(figure_size=(7.5, 5))
)
p_q2.show()
```



Parágrafo de achados (Questão 2)

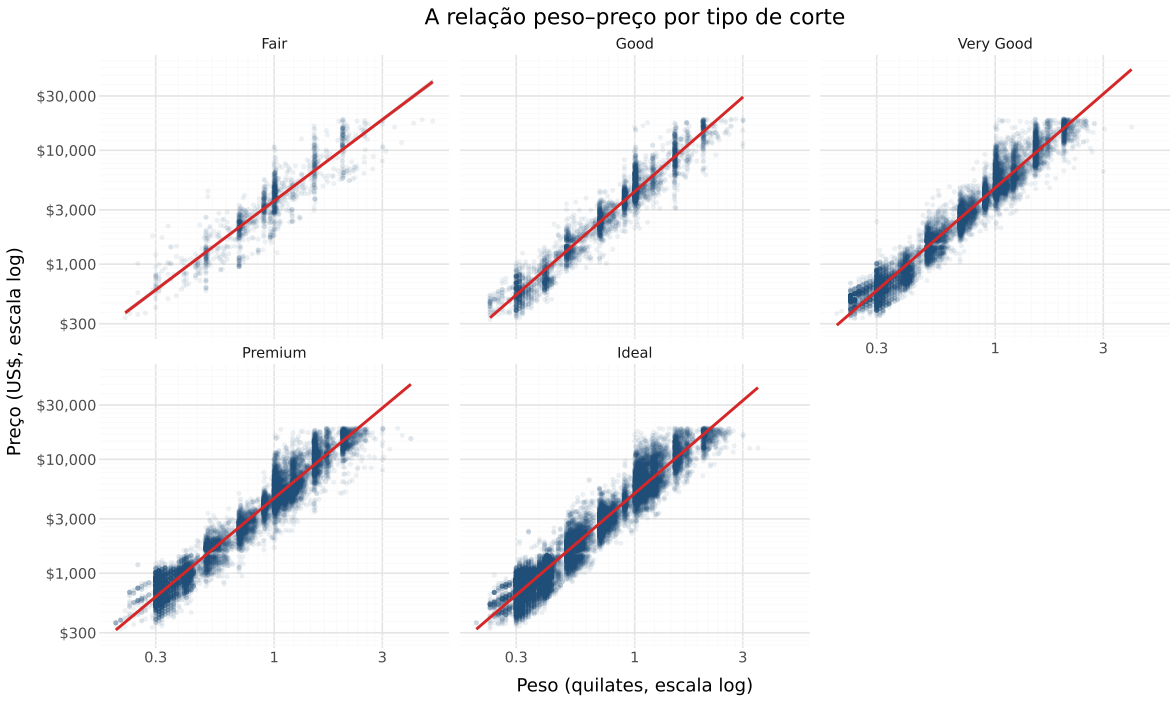
Ações para garantir qualidade: (i) testei primeiro o gráfico simples e identifiquei sobreposição de pontos e uma relação curva com variância crescente; (ii) usei transparência (`alpha`) e pontos menores para revelar a densidade; (iii) apliquei escala logarítmica nos dois eixos para linearizar a relação e estabilizar a variância; (iv) adicionei uma linha de tendência; (v) coloquei título, subtítulo, rótulos com unidades e um tema limpo.

Achados: existe uma associação **positiva e muito forte** entre peso e preço: quanto mais quilates, mais caro o diamante. A relação **não é linear** na escala original, mas fica aproximadamente **linear em escala log-log**, com inclinação maior que 1 (o preço cresce proporcionalmente mais que o peso) e correlação próxima de 1. Também se nota que há faixas verticais de pontos em pesos "redondos" (0,5; 1; 1,5; 2 ct), porque os diamantes tendem a ser lapidados e vendidos nesses pesos de referência. Por fim, para um mesmo peso há grande variação de preço, o que sugere que **outras características (corte, cor, clareza) também influenciam o valor**, tema das próximas questões.

3. A relação carat × price é a mesma em todos os cortes?

3.1 Um painel por corte

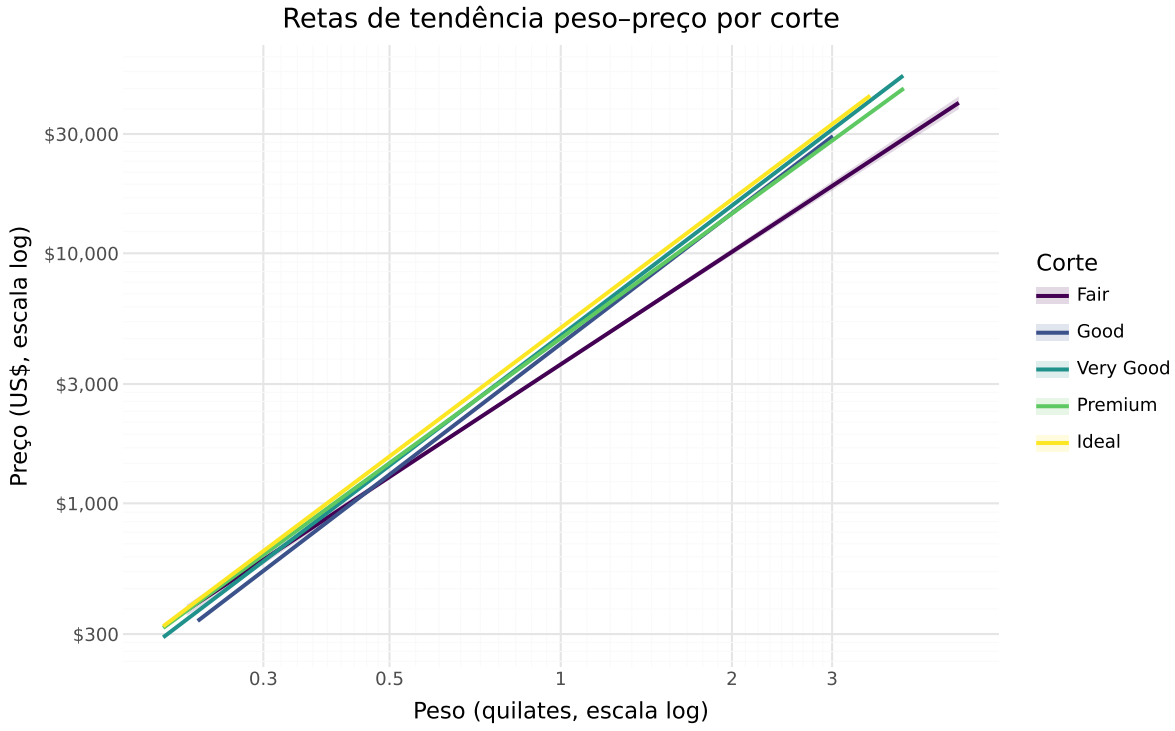
```
p_q3 = (
    ggplot(diamonds, aes(x="carat", y="price"))
    + geom_point(alpha=0.08, size=0.7, color="#1f4e79")
    + geom_smooth(method="lm", color="#d62728", size=1)
    + facet_wrap("~cut", nrow=2)
    + scale_x_log10(breaks=[0.3, 1, 3])
    + scale_y_log10(labels=fmt_dolar)
    + labs(
        title="A relação peso-preço por tipo de corte",
        x="Peso (quilates, escala log)",
        y="Preço (US$, escala log)",
    )
    + theme_minimal()
    + theme(figure_size=(10, 6))
)
p_q3.show()
```



3.2 Todas as retas juntas

Para comparar **inclinação e nível** das retas, colocamo-las no mesmo gráfico, uma cor por corte (sem pontos, para não poluir).

```
p_q3b = (
    ggplot(diamonds, aes(x="carat", y="price", color="cut", fill="cut"))
    + geom_smooth(method="lm", size=1.1, alpha=0.15)
    + scale_x_log10(breaks=[0.3, 0.5, 1, 2, 3])
    + scale_y_log10(labels=fmt_dolar)
    + scale_color_cmap_d(cmap_name="viridis", name="Corte")
    + scale_fill_cmap_d(cmap_name="viridis", name="Corte")
    + labs(
        title="Retas de tendência peso-preço por corte",
        x="Peso (quilates, escala log)",
        y="Preço (US$, escala log)",
    )
    + theme_minimal()
    + theme(figure_size=(8, 5))
)
p_q3b.show()
```



Parágrafo de conclusão (Questão 3)

Em todos os cortes a relação é **positiva e aproximadamente linear em escala log-log**, ou seja, o padrão geral "mais quilates → mais caro" se repete em cada grupo. Porém, ao comparar as retas, vemos que o **nível (intercepto) e a inclinação variam um pouco entre cortes** (veja a tabela acima): a relação **não é exatamente a mesma**, ainda que seja qualitativamente semelhante. Os painéis também mostram que os grupos têm tamanhos muito diferentes (poucos `Fair`, muitos `Ideal`) e cobrem faixas de peso distintas, o que deve ser considerado ao comparar. Conclusão: o corte **modifica** a relação (principalmente o patamar de preço para um mesmo peso), mas não a destrói.

4. Preço × peso considerando corte, cor e clareza ao mesmo tempo

Combinando as três variáveis categóricas temos $5 \times 7 \times 8 = \mathbf{280}$ **combinações possíveis**.

```
combos = (
    diamonds.groupby(["cut", "color", "clarity"], observed=True)
    .size()
    .reset_index(name="n")
)

print("Combinações existentes:", len(combos))
```

```
Combinações existentes: 276
```

```
print("Mínimo de diamantes em uma combinação:", combos["n"].min())
```

```
Mínimo de diamantes em uma combinação: 1
```

```
print("Mediana:", combos["n"].median())
```

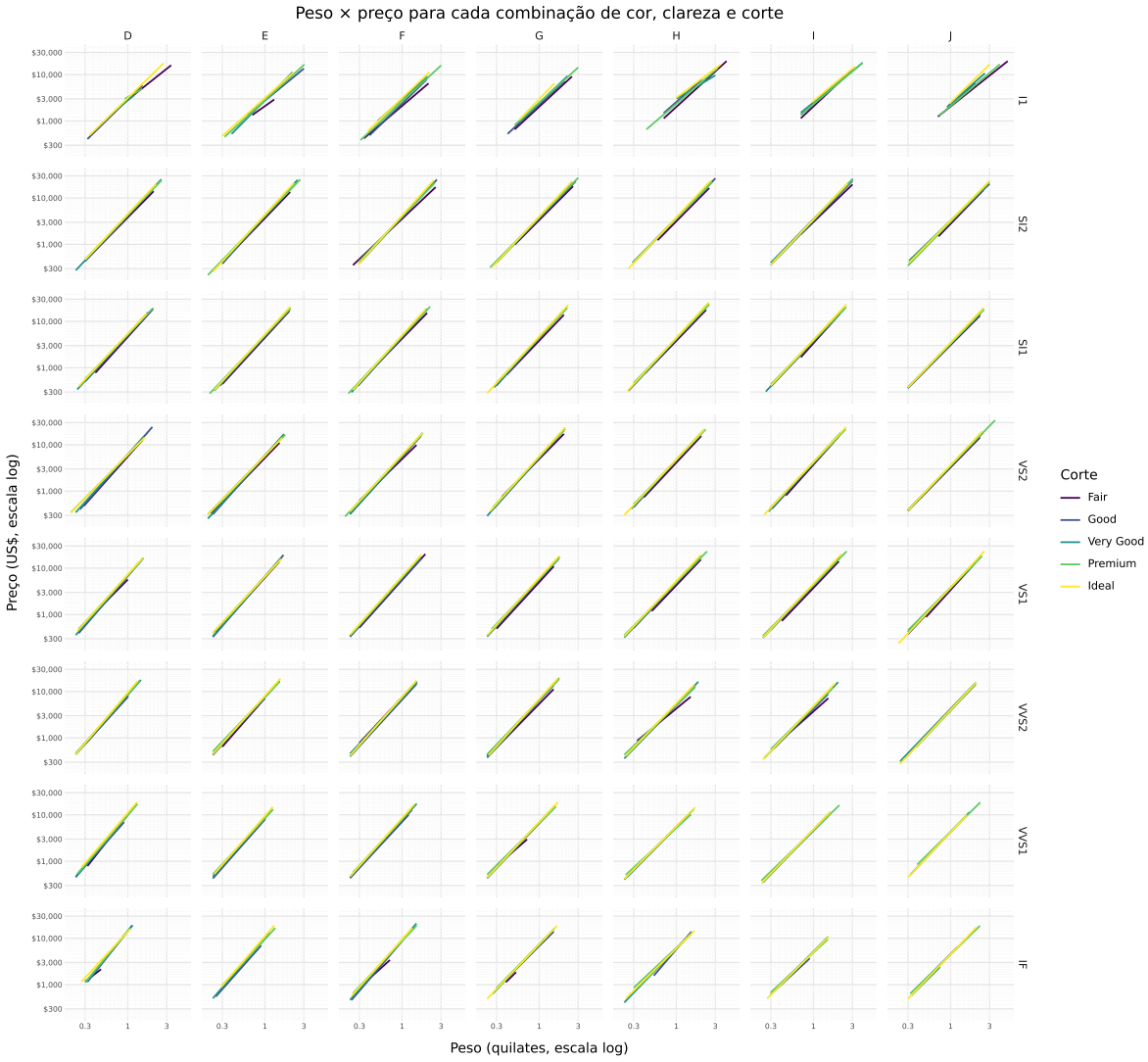
```
Mediana: 90.5
```

```
print("Combinações com menos de 30 diamantes:", (combos["n"] < 30).sum())
```

```
Combinações com menos de 30 diamantes: 77
```

4.1 Tentativa direta: facetas cor × clareza, retas coloridas por corte

```
p_q4a = (
    ggplot(diamonds, aes(x="carat", y="price", color="cut"))
    + geom_smooth(method="lm", se=False, size=0.8)
    + facet_grid("clarity ~ color")
    + scale_x_log10(breaks=[0.3, 1, 3])
    + scale_y_log10(labels=fmt_dolar)
    + scale_color_cmap_d(cmap_name="viridis", name="Corte")
    + labs(
        title="Peso × preço para cada combinação de cor, clareza e corte",
        x="Peso (quilates, escala log)",
        y="Preço (US$, escala log)",
    )
    + theme_minimal()
    + theme(figure_size=(13, 12), axis_text=element_text(size=6))
)
p_q4a.show()
```



4.2 Resumindo a informação: mapa de calor das inclinações

Resumimos cada combinação em um número (a inclinação log-log) e **filtramos** combinações com menos de 30 diamantes ou pouca variação de peso.

```
linhas = []
for (corte, cor, clar), g in diamonds.groupby(["cut", "color", "clarity"], observed=True):
    # exige n mínimo e variação de peso, senão a reta não é confiável
    if len(g) < 30 or g["carat"].nunique() < 5:
        continue
```

```
    b, a = np.polyfit(np.log10(g["carat"]), np.log10(g["price"]), 1)
    linhas.append({"cut": corte, "color": cor, "clarity": clar,
                   "inclinacao": b, "n": len(g)})

incl_ok = pd.DataFrame(linhas)

# Garante a ordem correta das categorias para o gráfico
incl_ok["cut"]     = pd.Categorical(incl_ok["cut"],     categories=ordem_cut,     ordered=Tru
incl_ok["color"]   = pd.Categorical(incl_ok["color"],   categories=ordem_color,   ordered=Tru
incl_ok["clarity"] = pd.Categorical(incl_ok["clarity"], categories=ordem_clarity, ordered=Tru

print(f"Combinações mantidas: {len(incl_ok)} de {len(combos)}")
```

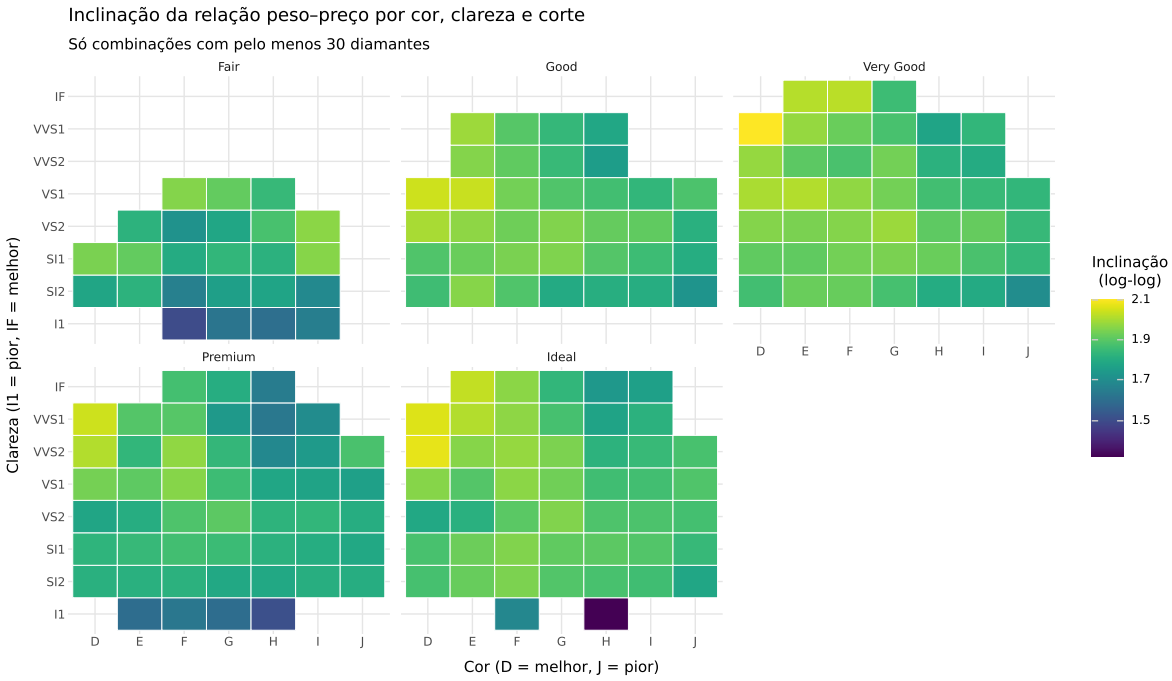
```
Combinações mantidas: 199 de 276
```

```
incl_ok["inclinacao"].describe().round(2)
```

```
count    199.00
mean       1.86
std        0.11
min        1.32
25%        1.81
50%        1.87
75%        1.94
max        2.11
Name: inclinacao, dtype: float64
```

```
p_q4b = (
    ggplot(incl_ok, aes(x="color", y="clarity", fill="inclinacao"))
    + geom_tile(color="white")
    + facet_wrap("~cut", nrow=2)
    + scale_fill_cmap(cmap_name="viridis", name="Inclinação\n(log-log)")
    + labs(
        title="Inclinação da relação peso-preço por cor, clareza e corte",
        subtitle="Só combinações com pelo menos 30 diamantes",
        x="Cor (D = melhor, J = pior)",
        y="Clareza (I1 = pior, IF = melhor)",
    )
    + theme_minimal()
    + theme(figure_size=(12, 7))
```

```
)
p_q4b.show()
```



4.3 Quanto o patamar de preço muda? (preço mediano por 1 ct)

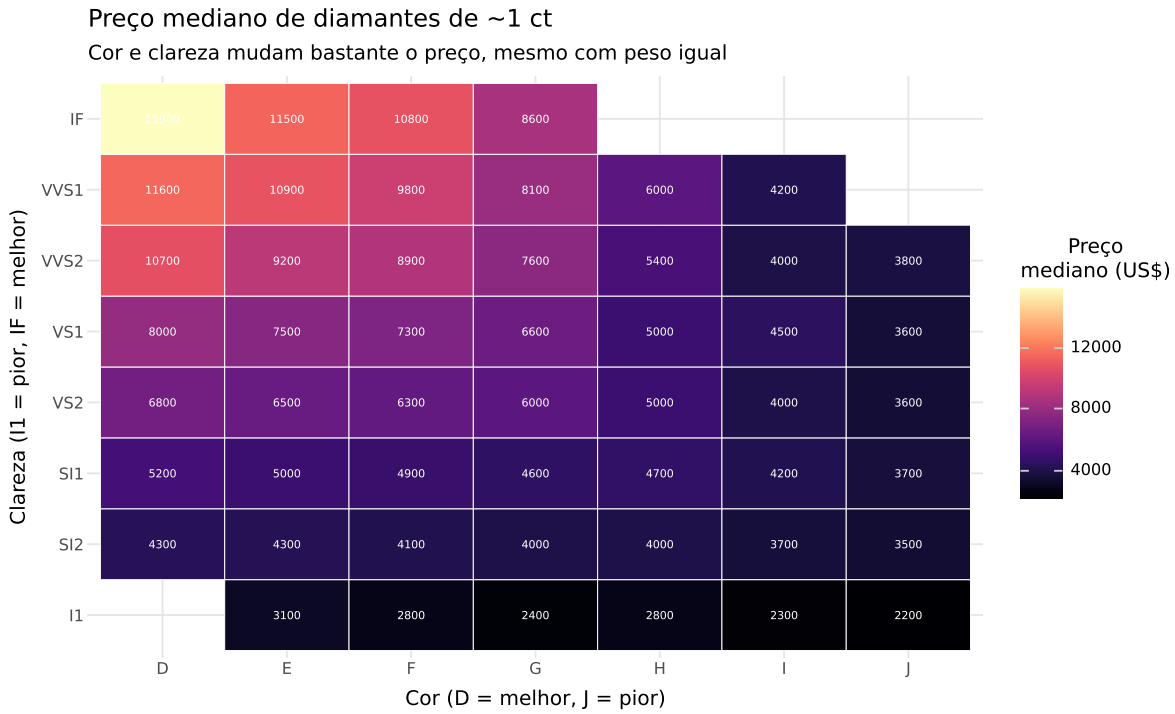
Comparamos o **nível** de preço de diamantes com peso parecido (0,9 a 1,1 ct), por cor e clareza, isolando o efeito da qualidade do efeito do peso.

```
um_carat = diamonds[(diamonds["carat"] >= 0.9) & (diamonds["carat"] <= 1.1)]

resumo = (
    um_carat.groupby(["color", "clarity"], observed=True)
    .agg(preco_mediano=("price", "median"), n=("price", "size"))
    .reset_index()
)
resumo = resumo[resumo["n"] >= 10].copy()

# Texto do rótulo calculado antes (mais seguro que expressões dentro do aes)
resumo["rotulo"] = (resumo["preco_mediano"] / 100).round().astype(int) * 100
resumo["rotulo"] = resumo["rotulo"].astype(str)
```

```
p_q4c = (
    ggplot(resumo, aes(x="color", y="clarity", fill="preco_mediano"))
    + geom_tile(color="white")
    + geom_text(aes(label="rotulo"), size=6, color="white")
    + scale_fill_cmap(cmap_name="magma", name="Preço\nmediano (US$)")
    + labs(
        title="Preço mediano de diamantes de ~1 ct",
        subtitle="Cor e clareza mudam bastante o preço, mesmo com peso igual",
        x="Cor (D = melhor, J = pior)",
        y="Clareza (I1 = pior, IF = melhor)",
    )
    + theme_minimal()
    + theme(figure_size=(9, 5.5))
)
p_q4c.show()
```



Dificuldades encontradas e conclusão (Questão 4)

A relação entre preço e peso é a mesma em todas as configurações? Em **direção**, sim: em praticamente todas as combinações, mais peso significa preço maior. Em **intensidade**

e nível, não: as inclinações variam entre combinações (mapa de calor) e, principalmente, o **patamar de preço** muda bastante com a cor e a clareza (último gráfico): para um mesmo peso, diamantes de melhor cor e clareza custam mais. Portanto, **peso é o principal determinante do preço, mas a qualidade (corte, cor e clareza) desloca e ajusta essa relação**; analisar o peso isoladamente esconde parte importante da história.

Informações da sessão (reprodutibilidade)

```
import sys, plotnine
print("Python:", sys.version.split()[0])
```

```
Python: 3.13.16
```

```
print("pandas:", pd.__version__)
```

```
pandas: 3.0.6
```

```
print("numpy:", np.__version__)
```

```
numpy: 2.5.4
```

```
print("plotnine:", plotnine.__version__)
```

```
plotnine: 0.15.8
```